

Version	0.18
Date	2026-08-10
Firmware	1.1.4

D21 eStation Developer Manual

This document applies only to the integration of ETAG ESL Gen 3.0 and DSL Gen 1.0 systems.

Technical Support: huanghaipengonline@hotmail.comhuanghaipengonline@hotmail.com

Revision History

Version	Date	Summary	Author	Reviewed by
0.1	2024-08-07	Initial document creation	Huang Haipeng	Huang Haipeng
0.2	2024-08-13	Added descriptions of device-side Topic QoS levels	Huang Haipeng	Huang Haipeng
0.3	2024-08-14	Corrected errors in the TaskResult data structure definition	Huang Haipeng	Huang Haipeng
0.4	2025-01-05	Added secure communication description and revised Pattern definition	Huang Haipeng	Huang Haipeng
0.5	2025-01-07	Added the Bytes-array-based push interface	Huang Haipeng	Huang Haipeng
0.6	2025-01-08	Added supplementary notes for the Bytes-array push interface	Huang Haipeng	Huang Haipeng
0.7	2025-02-17	Added Topic Alias Added DSL task and OTA interfaces	Huang Haipeng	Huang Haipeng
0.8	2025-03-01	Corrected the Pattern section and adjusted Topic names	Huang Haipeng	Huang Haipeng
0.9	2025-03-25	Corrected key communication errors	Huang Haipeng	Huang Haipeng
0.10	2025-04-10	Corrected the data structure definition for the DSL task interface Added RGB combination color table	Huang Haipeng	Huang Haipeng
0.11	2025-04-30	Added notes on steady-on mode for DSL tasks and OTA MD5 values	Huang Haipeng	Huang Haipeng
0.12	2025-06-23	Corrected data-type errors in data structure definitions	Huang Haipeng	Huang Haipeng
0.13	2025-07-26	Added the DSLv2 task interface and data structure definitions	Huang Haipeng	Huang Haipeng
0.14	2025-08-12	Added ESL image-size descriptions Corrected minor errors	Huang Haipeng	Huang Haipeng
0.15	2025-08-19	Corrected minor errors Removed the DSL Pattern.UpdateImage command	Huang Haipeng	Huang Haipeng
0.16	2026-04-13	Added support for 6-color and 7-color ESLs Added support for mainstream image formats	Huang Haipeng	Huang Haipeng
0.17	2026-05-07	Added ESL image-size descriptions Corrected minor errors	Huang Haipeng	Huang Haipeng
0.18	2026-08-10	Added mTLS support	Huang Haipeng	Huang Haipeng

Table of Contents

1	Introduction.....	4
1.1	Background	4
1.2	eStation System Architecture.....	6
1.3	About Images.....	6
2	Using eStation	8
2.1	Connecting to eStation.....	9
2.2	Receiving eStation Information.....	10
2.3	Receiving eStation Messages.....	11
2.4	Receiving eStation Results	11
2.5	Receiving eStation Heartbeats	12
2.6	Publishing Configuration Information	13
2.7	Publishing ESL Tasks (Base64 Version)	14
2.8	Publishing ESL Tasks (Bytes Version).....	15
2.9	Publishing DSL Tasks	16
2.10	Publishing DSL Tasks (with Pattern and Key).....	16
2.11	Publishing OTA Tasks (Firmware)	17
2.12	Publishing OTA Tasks (ESL/DSL)	18
2.13	Publishing Certificate Import Tasks.....	18
2.14	Secure Communication	19
3	Reference	20
3.1	ESL Gen 3.0 Model List	20
3.2	Pattern.....	21
3.3	PageIndex.....	21
3.4	RGB Color Definition.....	22
3.5	Data Definitions (C# Version)	22

1 Introduction

1.1 Background

The eStation IoT device, type code AP05, is designed for system integrators to rapidly develop their business projects using the MQTT v5 protocol.

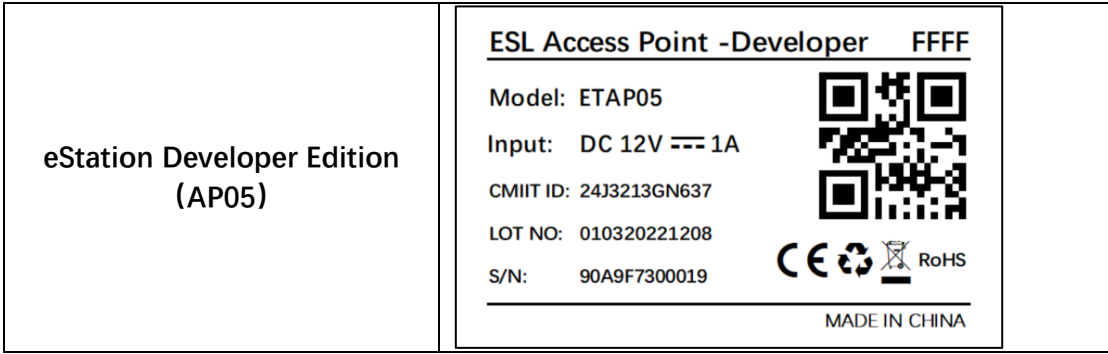
Difference between AP04 and AP05: Before using eStation, please carefully review the differences between eStation AP04 and AP05. Both use MQTT as the communication protocol, but their protocol topics and data structure definitions differ slightly.

Difference between the AP05 Standard Edition and Developer Edition: The AP05 Developer Edition provided for system integration and the Standard Edition provided for eRetail 3.2/4.0 both use the same four-character ID scheme, but their protocol topics and data structure definitions are different. This document covers only the Developer Edition.

You can distinguish them by the nameplate on the back of the device.

The appearance differences among AP04 Developer Edition, AP05, and AP05 Developer Edition are shown below:

eStation Developer Edition (AP04)	ESL Access Point Model:ETAP04 Input:DC 12V  3A CMIIT ID:2022DP10012 FCC ID:2ARJ5-ETAP04   211-221123 LOT NO:010120240307 SN/MAC:18BC5A661234  MADE IN CHINA eStation Developer Edition Default Server Address: 192.168.1.92:9080 Default User Name: test Default Password: 123456	
eStation (AP05)	ESL Access Point 003T Model: ETAP05 Input: DC 12V  1A CMIIT ID: 24J3213GN637 LOT NO: P20231220000133 S/N: 90A9F7400081    RoHS MADE IN CHINA	



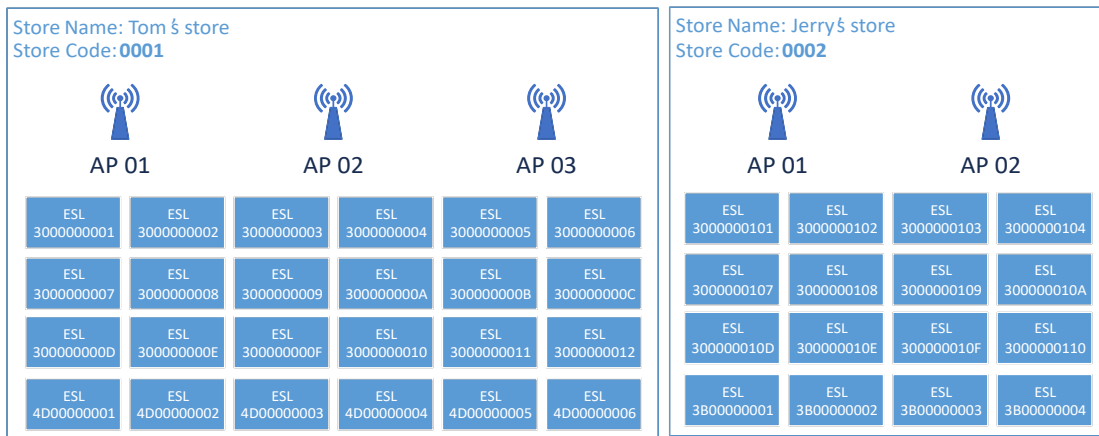
A demo project for eStation Developer Edition is available on GitHub at:
https://github.com/andersonhwang/eStation_Developer_Edition.

Note: This project is still under development. The .NET Console and WPF MVVM versions have currently been completed.

Before using eStation, you should understand the following key points:

1. **AP: A radio-frequency access point connected to ESLs.**
2. **AP ID: AP05 uses a four-character ID as its globally unique identifier. The ID can be found on the nameplate attached to the back of the device.**
3. **ESL: Electronic Shelf Label.** In this document, it specifically refers to ESL Gen 3.0 using Bluetooth Low Energy (BLE 5.0).
4. **DSL: A rail-powered electronic shelf label, currently limited to the 2.4-inch TFT model.**
5. **MQTT: eStation (AP05) uses the MQTT communication protocol.**
6. **Base64String: Converts an image to a Base64String. For more information, refer to Base64 Encode and Decode - Online. [Base64 Encode and Decode - Online](#)**
7. **MessagePack: eStation (AP05) uses the MessagePack data format to reduce packet size. For more information, refer to MessagePack: It's like JSON, but fast and small (msgpack.org).[MessagePack: It's like JSON. but fast and small. \(msgpack.org\)](#)**

For example, suppose your project has two stores, as shown below:



The data structure should be:

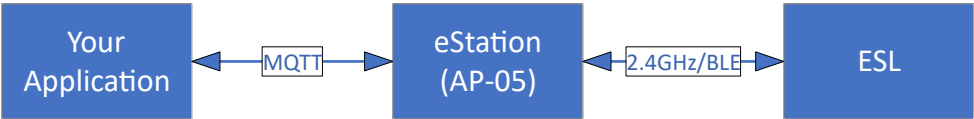
Store Code	AP ID	ESL ID
0001	01	300000000001, 300000000002, 300000000007...

	02	3000000000003, 3000000000004, 3000000000009 ...
	03	3000000000005, 3000000000006, 300000000000B...
0002	01	3000000000101, 3000000000102, 3000000000107...
	02	3000000000103, 3000000000104, 3000000000109...

Note: Normally, your application should remember the AP that successfully communicated with each ESL. Physically, however, an ESL is not bound to any AP. The ESL only logically attempts to remember the ID of the last default AP that successfully sent data to it.

1.2 eStation System Architecture

In general, your project sends tasks containing an ESL ID and image data (Base64String) to eStation. eStation uses 2.4 GHz radio frequency (RF) to send the image data to the specified ESL, and then returns the result to your project. In other words, eStation sits between your application and the ESL.



This document describes two sides: your application side and the eStation device side.

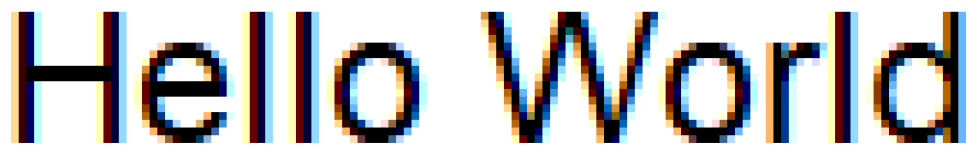
If your application and eStation are not on the same private network, use a strong password to protect the connection and add an X.509 certificate.

1.3 About Images

Electronic ESL screens are monochrome or black/white/red (yellow) and do not support grayscale. Suppose your image looks like this:

Hello World

At 800% magnification, it looks like this:



After removing the grayscale portion, it looks like this:

Hello World

The larger the font, the more visible the jagged edges become.

Using ESLs to display photographic images is not recommended. If images must be displayed, developers should apply dithering to simulate grayscale.

An example of a C# image-dithering algorithm is: [Even more algorithms for dithering images using C# - Articles and information on C# and .NET development Topics • Cyotek](#)

For example, an image may look like this:



The results with and without dithering are shown below:



2 Using eStation

As described above, there is no code-level integration with eStation. Your application should manage the connection and communication.

There are 10 topics: four are published by the eStation device, and six are published by the application.

From the application side, the eStation device subscribes to the following topics (Topic Aliases):

1. **config (0x01):** Use this topic to configure eStation.
2. **taskESL (0x02):** Publish ESL tasks with image data in Base64 string format. This topic is deprecated and retained only for backward compatibility.
3. **taskESL2 (0x03):** Publish ESL tasks with image data as a byte array. This topic is recommended.
4. **taskDSL (0x04):** Publish DSL tasks with this topic (currently limited to the 2.4-inch rail-powered TFT label).
5. **firmware (0x05):** Use this topic to perform OTA updates for eStation, or to

preload ESL/DSL firmware for use by the ota topic.

6. **ota (0x06):** Use this topic to perform OTA updates for ESLs and DSLs.

7. **cert (0x08):** Use this topic to update certificate files on eStation side.

From the eStation device side, your application should subscribe to the following topics (Topic Aliases):

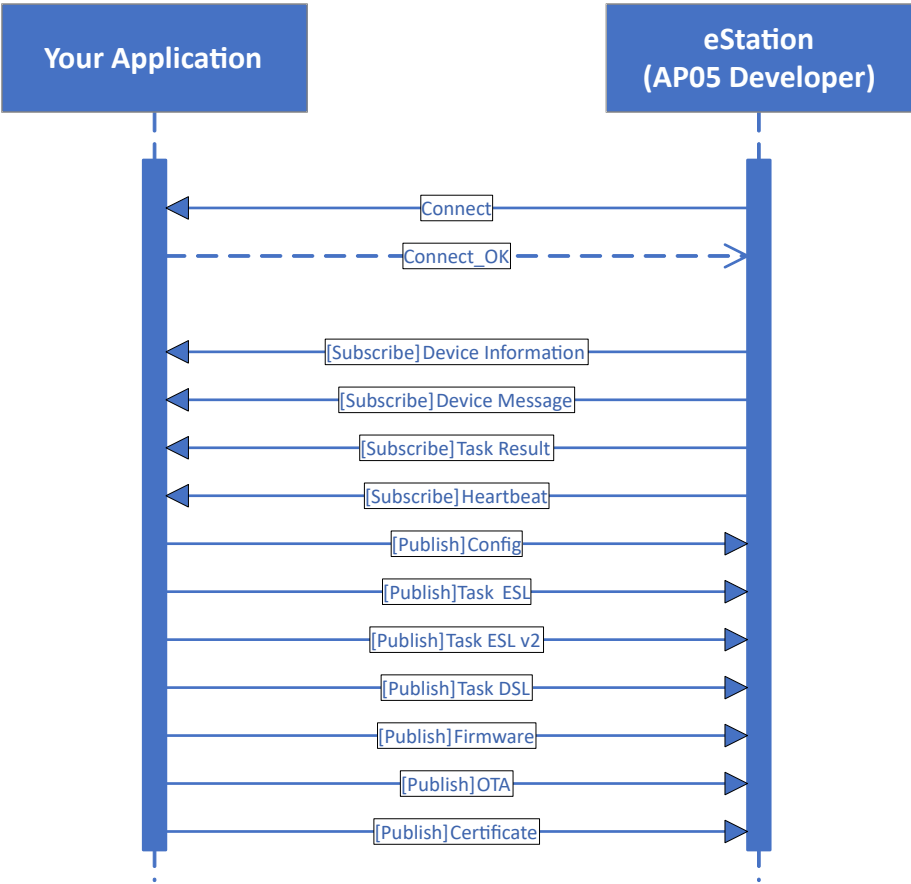
1. **infor (0x80):** After eStation establishes a connection to the server, it reports device information.

2. **message (0x81):** eStation returns message codes when events occur.

3. **result (0x82):** eStation returns a result after communicating with an ESL.

4. **heartbeat (0x83):** eStation periodically reports its current status.

Note: You may omit the Topic Alias (default: 0). If a Topic Alias is specified, it takes precedence.



Note: Publish/Subscribe is from your application side

2.1 Connecting to eStation

eStation operates as an IoT client device and attempts to connect to your application (server side) after power-on.

The default eStation parameters are:

Default target server address	192.168.1.92:9081
Default username	test
Default password	123456

Note: Change the username and password as soon as possible.

After connecting to an eStation, you can modify its settings using the configuration topic (see Section 2.6, Publish Configuration Information). 2.7 Publish Configuration Information

If you cannot connect to eStation, or if you have forgotten its target server address or username/password, press and hold the reset button for several seconds. When REST and the boot indication appear, the target server IP address, username, and password will be restored to their defaults.

Note: If eStation still cannot connect to your application after a reset, check the firewall and Ethernet cable connection.

2.2 Receiving eStation Information

Interface Description:

Topic Alias	0x80
Topic	{Prefix}/{ID}/infor
Prefix	Topic prefix; default is "/estation" (without quotation marks)
ID	eStation ID
QoS Level	ExatclyOnce
PayloadSegment	eStationInfor object bytes, MessagePack serialized

The eStationInfor properties are:

#	Property	Type	Notes
0	ID	Char (4)	Unique four-character ID
1	Alias	String	Alias; if it is a two-digit number, it is shown on the digital display
2	IP	IP String	Local IP address
3	MAC	MAC String	MAC address
4	ApType	Int	Fixed at 6
5	ApVersion	String	Main application firmware version
6	ModVersion	String	Module firmware version
7	DiskSize	Int	Disk size, MB
8	FreeSpace	Int	Free space, MB
9	Server	String	Target server address; may be an IP address or domain name
10	ConnParam	String array	Connection parameters
11	Encrypt	Bool	True: enable TLS 1.2; False: disable it Disabled by default; see X.509 Certificate。 X.509 Certificate X.509 Certificate
12	AutoIP	Bool	Obtain IP automatically
13	LocalIP	IP String	Static IP address
14	Subnet	IP String	Subnet mask
15	Gateway	IP String	Gateway
16	Heartbeat	Int	Heartbeat interval in seconds; minimum 15

Note: When Encrypt is True, if certificates were written to the device using 0x08

before configuration, the device enters mTLS mode after configuration; the server must also support mTLS. If the device detects no usable certificate, it enters TLS mode.

2.3 Receiving eStation Messages

Interface Description:

Topic Alias	0x81
Topic	/estation/{ID}/message
ID	eStation ID
QoS Level	AtMostOnce
PayloadSegment	eStationMessage object bytes, MessagePack serialized

The eStationMessage properties are:

#	Property	Type	Notes
0	Code	MessageCode	Enum type

The MessageCode enumeration values are:

0. OK General response
1. Idle
2. Result Communication result returned
3. Heartbeat Heartbeat monitoring feedback
4. ModError Communication module error
5. AppError Main application error
6. Busy Device busy
7. MaxLimit The current data queue has reached its limit (more than 10,000 pending items or memory usage exceeds 512 MB)
8. InvalidTaskESL Invalid ESL task data
9. InvalidTaskDSL Invalid DSL task data
10. InvalidConfig Invalid configuration data
11. InvalidOTA Invalid OTA data

2.4 Receiving eStation Results

Interface Description:

Topic Alias	0x82
Topic	/estation/{ID}/result
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	TaskResult object bytes, MessagePack serialized TaskResult contains a list of TagResult objects

The TaskResult properties are:

#	Property	Type	Notes
0	Port	Int	MOD serial port number
1	WaitCount	Int	Total number of tasks in the cache
2	SendCount	Int	Number of tasks being sent (in the RF module)

3	Message	String	Reserved field
4	Tags	List<TagResult>	ESL result list

The TagResult properties are:

#	Property	Type	Notes
0	TagID	String	ESL ID
1	RfPower	Int	RF power, dBm; -256 means no data
2	Battery	Byte	Battery voltage, V; 0 means no data
3	Version	Byte	ESL firmware version; reserved
4	Status	Byte	Status; reserved
5	Token	Int	Token used for closed-loop data tracking
6	Temperature	Int	Temperature, °C; 0 means no data
7	Channel	Int	Channel; reserved
8	UtcTime	Byte array	UTC time; reserved; DSL only
9	TimePercent	Byte array	Time confidence; reserved; DSL only
10	Count	Byte	Counter; reserved; DSL only

Note: eStation maintains an internal queue. When the application sends task data, the task is sent to the RF module immediately if the RF module is idle; otherwise, it waits until the RF module becomes idle. Therefore, communication between the application and eStation is asynchronous.

2.5 Receiving eStation Heartbeats

Interface Description:

Topic Alias	0x83
Topic	/estation/{ID}/heartbeat
ID	eStation ID
QoS Level	AtMostOnce
PayloadSegment	ApHeartbeat object bytes, MessagePack serialized ApHeartbeat contains a list of TagHeartbeat objects

The ApHeartbeat properties are:

#	Property	Type	Notes
0	Id	String	AP ID
1	ConfigVersion	Int	Configuration version (default 0; reserved)
2	ApVersion	String	AP version
3	ModVersion	String	Bluetooth module version
4	Message	MessageCode	Message; see MessageCode
5	MessageEx	String	Message (extended field)
6	WaitCount	Int	Number of ESLs currently queued
7	SendCount	Int	Number of ESLs currently communicating
8	Tags	TagHeartbeat list	ESL heartbeat information

The TagHeartbeat properties are:

#	Property	Type	Notes
0	TagId	String	ESL/DSL ID
1	RfPower	Int	RF power, dBm; -256 means no data

2	Battery	Byte	Battery voltage, V; 0 means no data
3	Version	Byte	ESL firmware version; reserved
4	Status	Byte	Status; reserved
5	Token	Int	Token used for closed-loop data tracking
6	Temperature	Int	Temperature, °C; 0 means no data
7	Channel	Int	Channel; reserved
8	UtcTime	Byte array	UTC time; DSL only
9	TimePercent	Byte	Time confidence; DSL only
10	Count	Byte	Counter; DSL only
11	Factory	Byte	Factory code for OTA
12	Color	Byte	Screen color code for OTA
13	Size	Byte	Screen size code for OTA
14	Type	Byte	Screen type code for OTA

Note: The heartbeat and result data structures are similar. A heartbeat contains heartbeat information received from labels, whereas a result is returned after your application sends data. The former occurs proactively; the latter occurs in response to a task.

2.6 Publishing Configuration Information

Interface Description:

Topic Alias	0x01
Topic	/estation/{ID}/ configure
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	eStationConfig object bytes, MessagePack serialized

The eStationConfig properties are:

#	Property	Type	Notes
0	Alias	String	Alias; if it is a two-digit number, it is shown on the digital display
1	Server	String	Target server address
2	ConnParam	String array	Connection parameters
3	Encrypt	Bool	True: enable TLS 1.2; False: disable it Disabled by default; see X.509 CertificateX.509 Certificate 2.9.2 X.509 Certificate
4	AutoIP	Bool	Obtain IP automatically
5	LocalIP	IP String	Static IP address
6	Subnet	IP String	Subnet mask
7	Gateway	IP String	Gateway
8	Heartbeat	Int	Heartbeat interval in seconds; minimum 15

2.7 Publishing ESL Tasks (Base64 Version)

Note:

1. This topic is deprecated. The Bytes-version topic in Section 2.8 is recommended.
2. Only 3-color (Black-White-Red) and 4-color (Black-White-Red-Yellow) ESLs are supported.

Interface Description:

Topic Alias	0x02
Topic	/estation/{ID}/taskESL
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	An array/list of ESLEntity objects, MessagePack serialized

The ESLEntity properties are:

#	Property	Type	Notes
0	TagID	String	ESL ID
1	Pattern	Int, Pattern	Pattern code; see Section 3.2 Pattern 3.1 Pattern
2	PageIndex	Int, PageIndex	Page index; see Section 3.3 PageIndex 3.2 PageIndex
3	R	Bool	Red LED
4	G	Bool	Green LED
5	B	Bool	Blue LED
6	Times	Int, 2-byte range	Number of LED flashes
7	Token	Int, 2-byte range	Token, returned in the result 2.3 Token
8	CurrentKey	Int, 8-byte range	Current key; leave empty to use the default key
9	NewKey	8bytes	New key; leave empty if no change is required
10	Base64String	String	Base64String value of the image in Bitmap format

Overwrite: For an ESL ID, if your application sends another task without waiting for the result of the previous task, eStation discards the previous task.

Pixels: Pixel data must be in BGRA format; otherwise, color parsing errors or abnormal display may occur.

Size: eStation checks the size (length) of the image data (Base64String) in an ESL task. If the length is incorrect, or the actual image dimensions do not match the ESL screen dimensions/padded dimensions, the image cannot be updated successfully. The actual image dimensions cannot be inferred from the byte-array length alone; for example, 300×400 and 200×600 bitmap images have the same data length.

For the relationship between ESL model and screen resolution, see Section 3.1, ESL Gen 3.0 Model List.[ESL Gen 3.0 Model List](#)

In particular, because screen data width is processed in multiples of 8, some models may not meet this requirement. You may send data using the actual screen dimensions or pad the width to a multiple of 8; the extra portion is neither processed nor displayed. For example, an ET0213 accepts both 250×122 and 256×122 images, but only the 250×122 portion is displayed.

Limit: When data is pushed using Base64 strings, the eStation task-cache limit is relatively low (approximately 800 tasks in testing with a 4.2-inch 3-color ESL). To increase the limit, use the Bytes-version task interface.

Key: For more information about the current key and new key, see [Secure Communication](#).[Secure Communication](#)[2.12 Secure Communication](#)

2.8 Publishing ESL Tasks (Bytes Version)

Interface Description:

Topic Alias	0x03
Topic	/estation/{ID}/taskESL2
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	An array/list of ESLEntity2 objects, MessagePack serialized

The ESLEntity2 properties are:

#	Property	Type	Notes
0	TagID	String	ESL ID
1	Pattern	Int, Pattern	Pattern code; see Section 3.2 Pattern 3.1 Pattern
2	PageIndex	Int, PageIndex	Page index; see Section 3.3 PageIndex 3.2 PageIndex
3	R	Bool	Red LED
4	G	Bool	Green LED
5	B	Bool	Blue LED
6	Times	2bytes	Number of LED flashes
7	Token	2bytes	Token, returned in the result 2.3 Token
8	CurrentKey	8bytes	Current key; leave empty to use the default key
9	NewKey	8bytes	New key; leave empty if no change is required
10	Bytes	Byte array	Image data (see below)
11	Compress	Bool	Whether to compress; default True; GZip compression

Pixels: The image-pixel requirements for this topic are the same as those for Publishing ESL Tasks (Base64 Version). Publishing ESL Tasks (Base64 Version)[2.7 Publishing ESL Tasks \(Base64 Version\)](#)

Size: The image-size requirements for this topic are the same as those for Publishing ESL Tasks (Base64 Version). Publishing ESL Tasks (Base64 Version)[2.7 Publishing ESL Tasks \(Base64 Version\)](#)

Limit: With the Bytes-version topic, the current eStation limit is 20,000 tasks, which is sufficient for most scenarios. Tasks exceeding this limit are discarded, and MessageCode.MaxLimit is returned through the message topic.

Image data: Bitmap pixel arrays in BGRA32 and BGR24 formats are supported, as are image files such as BMP, JPEG, and PNG. GZip compression is supported; set Compress to True when using it. Unlike taskESL, image dimensions must strictly match the screen resolution defined for the product model.

2.9 Publishing DSL Tasks

Interface Description:

Topic Alias	0x04
Topic	/estation/{ID}/taskDSL
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	An array/list of DSLEntity objects, MessagePack serialized

The DSLEntity properties are:

#	Property	Type	Notes
0	TagID	String	ESL ID
1	R	Bool	Red LED
2	G	Bool	Green LED
3	B	Bool	Blue LED
4	Period	2bytes	LED flash count, -1 to 3600; default 3600
5	Interval	2bytes	LED interval, 100 to 10000; default 1000 ms
6	Duration	8bytes	LED on-time, 50 to 100; default 50 ms
7	Token	8bytes	Token, returned in the result 2.3 Token
8	HexData	Byte array	Bin file data stream

Data: A Bin file is an image/command file generated specifically for the 2.4-inch TFT label (DSL).

Steady-on: When Period = -1, the LED remains continuously on; Interval and Duration are ignored.

2.10 Publishing DSL Tasks (with Pattern and Key)

Interface Description:

Topic Alias	0x07
Topic	/estation/{ID}/taskDSL2
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	An array/list of DSLEntity2 objects, MessagePack serialized

The DSLEntity2 properties are:

#	Property	Type	Notes
0	TagID	String	ESL ID

1	R	Bool	Red LED
2	G	Bool	Green LED
3	B	Bool	Blue LED
4	Period	2bytes	LED flash count, -1 to 3600; default 3600
5	Interval	2bytes	LED interval, 100 to 10000; default 1000 s
6	Duration	8bytes	LED on-time, 50 to 100; default 50 ms
7	Token	8bytes	Token, returned in the result 2.3 Token
8	HexData	Byte array	Bin file data stream
9	Pattern	Int, Pattern	Pattern code; see Section 3.2 Pattern 3.1 Pattern
10	CurrentKey	8bytes	Current key; leave empty to use the default key
11	NewKey	8bytes	New key; leave empty if no change is required

Data: A Bin file is an image/command file generated specifically for the 2.4-inch TFT label (DSL).

Steady-on: When Period = -1, the LED remains continuously on; Interval and Duration are ignored.

2.11 Publishing OTA Tasks (Firmware)

Interface Description:

Topic Alias	0x05
Topic	/estation/{ID}/firmware
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	An array/list of OTADData objects, MessagePack serialized

The OTADData properties are:

#	Property	Type	Notes
0	DownloadUrl	String	Firmware download URL
1	ConfirmUrl	String	Target server address
2	Type	Int	OTA type: 0-eStation, 1/2-MOD, 3-ESL/DSL
3	Version	String	Version number
4	Name	String	Firmware name
5	MD5	IP String	Static IP address

Note:

1. If Type = 0, this topic treats the file as eStation firmware. After download and verification, eStation immediately restarts and performs the OTA update. If Type = 1 or 2, the file is treated as MOD firmware and the MOD OTA update starts immediately. If Type = 3, the file is stored as ESL/DSL firmware for later ESL/DSL OTA use.
2. After an OTA update starts, do not communicate with eStation until the OTA process has definitely completed.
3. MD5 format: In earlier versions (<1.0.28), the format is 00-11-22-33-44-55-66-77.

In later versions ($\geq 1.0.28$), the format is 00111223344556677.

2.12 Publishing OTA Tasks (ESL/DSL)

Interface Description:

Topic Alias	0x06
Topic	/estation/{ID}/ota
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	An array/list of OTATask objects, MessagePack serialized

The OTATask properties are:

#	Property	Type	Notes
0	Step	Int	Type: 0-PreOTA; 1-OTA; 2-Clean
1	Firmware	String	Firmware file name
2	TagIDList	String array	List of ESL IDs to update

Note:

1. This topic depends on the firmware described in the preceding firmware section; otherwise, the firmware cannot be found and the update cannot be performed.
2. This topic must be executed at least three consecutive times (Step = 0, 1, and 2 in sequence). Use the results described in Section 2.4 to verify that all ESLs in the current step communicated successfully before proceeding to the next step.
3. The TagIDList in this topic must contain ESL IDs of the same model, and Firmware must be the two-character code for that model.

2.13 Publishing Certificate Import Tasks

Interface Description:

Topic Alias	0x08
Topic	/estation/{ID}/cert
ID	eStation ID
QoS Level	ExactlyOnce
PayloadSegment	One ApCert object, MessagePack serialized

The ApCert properties are:

#	Property	Type	Notes
0	DownloadUrl	String	Download URL
1	ConfirmUrl	String	Confirm URL
2	CertName	String	Client certificate name
3	KeyName	String	Client key name
4	CustomTrustStore	String array	Custom trust store certificate(s)
5	ExtraStore	String array	Extra store certificate(s)
6	MD5	String	MD5

Note:

1. This topic only imports certificates into the device; it does not enable mTLS. To

enable mTLS, after pushing the certificates with this topic, set Encrypt = true using the Config topic.

2. Generating the client certificate and private key on the server and then downloading them to the device is not the preferred practice. For security, access to DownloadUrl should be one-time and its validity period should be short.

2.14 Secure Communication

2.12.1. ESL Key

In general, if another project uses its eStation (AP) to control the ESLs in your project, unexpected and unacceptable results may occur. In this case, it is recommended that you set a key for your ESLs.

When you receive new ESLs from the sales manager, the default ESL key is empty (11 bytes, represented as FFFFFFFFFFFFFFFF). You may keep it or set the current key and a new key. Subsequent communication should use the new key.

1. The key has a fixed length of 22 hexadecimal characters (11 bytes).
2. Only hexadecimal characters (0123456789ABCDEF) are accepted.
3. If the current key is empty, eStation substitutes the default key.
4. If New Key is empty, the AP will not change the key. If New Key is not empty, the AP will change the key.
5. **Important: Store your key securely. If you forget the key, you will lose control of the ESL.**
6. Only ESLs with firmware versions later than 35 support key-based communication.

2.12.2. X.509 Certificate

If TLS 1.2 is enabled in the configuration, the server can use a custom X.509 certificate to encrypt communication. How the server uses X.509 certificates and TLS 1.2 depends on the server-side development language and framework selected by the developer.

If the server is developed in C# using MQTTnet, a PFX certificate is required. Example code is shown below:

```
public async Task Run()
{
    var options = new MqttServerOptionsBuilder()
        .WithDefaultEndpoint()
        .WithEncryptedEndpoint()
        .WithEncryptedEndpointPort(PORT) // TCP port in your server side
        .WithEncryptionCertificate(CERTIFICATE_PATH) // Your certificate path
        .WithEncryptionSslProtocol(SslProtocols.Tls12)
        .Build();
    _server = _factory.CreateMqttServer(options);
    _server.ValidatingConnectionAsync += ServerOnValidatingConnectionAsync;
    _server.ClientConnectedAsync += ServeOnClientConnectedAsync;
    _server.ClientDisconnectedAsync += ServerOnClientDisconnectedAsync;
```

```

_server.InterceptingPublishAsync +=
    ServerOnInterceptingPublishAsync;
await _server.StartAsync();
}

```

3 Reference

3.1 ESL Gen 3.0 Model List

Model	Size (inches)	Display	Color	Pixels (H*W)	Freezer
ET0154-33	1.54	Eink	B/W/R	200*200	-
ET0213-36	2.13	Eink	B/W/R	250*122	-
ET0213-39	2.13	Eink	B/W	250*122	Y
ET0266-3A	2.66	Eink	B/W/R	296*152	-
ET0266-5B	2.66	Eink	B/W	296*152	Y
ET0290-3D	2.9	Eink	B/W/R	296*128	-
ET0290-3F	2.9	Eink	B/W	296*128	-
ET0290-54	2.9	Eink	B/W	296*128	Y
ET0420-40	4.2	Eink	B/W/R	400*300	-
ET0420-42	4.2	Eink	B/W	400*300	-
ET0750-44	7.5	Eink	B/W/R	800*480	-
ET0750-46	7.5	Eink	B/W	800*480	-
ET0430-4C	4.3	Eink	B/W/R	522*152	-
ET0580-4F	5.8	Eink	B/W/R	648*480	-
ET0350-55	3.5	Eink	B/W/R	384*184	-
ET1250-58	12.5	Eink	B/W/R	1304*984	-
ET0420-5D	4.2	TFT	B/W/R	400*300	-
ET1020-64	10.2	Eink	B/W/R	960*640	-
ET1020-67	10.20	Eink	B/W	960*640	-
ET1330-68	13.3	Eink	B/W/R	960*680	-
ET0579-6F	5.8	Eink	B/W/R	792*272	-
ET0154H-71	1.54	Eink	B/W/R	200*200	-
ET0130H-73	1.3	Eink	B/W/R	200*144	-
ET0154-80	1.54	Eink	B/W/R/Y	200*200	-
ET0213-81	2.13	Eink	B/W/R/Y	250*122	-
ET0266-82	2.66	Eink	B/W/R/Y	296*152	-
ET0266-83	2.66	Eink	B/W/R/Y	360*184	-
ET0290-84	2.9	Eink	B/W/R/Y	296*128	-
ET0290-85	2.9	Eink	B/W/R/Y	384*168	-

ET0350-86	3.5	Eink	B/W/R/Y	384*184	-
ET0420-87	4.2	Eink	B/W/R/Y	400*300	-
ET0580-88	5.8	Eink	B/W/R/Y	648*480	-
ET0750-89	7.5	Eink	B/W/R/Y	800*480	-
ET0750-8A	7.5	Eink	B/W/R/Y	880*528	-
ET1020-8B	10.2	Eink	B/W/R/Y	960*640	-
ET0097H-8C	0.97	Eink	B/W/R/Y	184*88	-
ET0130H-8D	1.3	Eink	B/W/R/Y	200*144	-
ET1330-8E	13.3	Eink	B/W/R/Y	960*680	-
ET0370-8F	3.7	Eink	B/W/R/Y	416*240	-
ET0154H-90	1.54	Eink	B/W/R/Y	200*200	-
ET0430-91	4.3	Eink	B/W/R/Y	522*152	-
ET0213-95	2.13	Eink	4-Color	250*122	Y
ET0266-96	2.66	Eink	4-Color	296*152	Y
ET0290-97	2.9	Eink	4-Color	296*128	Y
ET0600-99	5.8	Eink	B/W/R/W	792*272	-
ET0730-A0	7.3	Eink	6-Color	800*480	-
ET1330-A1	13.3	Eink	6-Color	1600*1200	-
ET0240C-D0	2.4	TFT	Full Color?	320*240	-

Note:

1. In the Color column, B means black, W means white, R means red, and Y means yellow.
2. In the Color column, 6-Color means black, white, red, yellow, green, and blue.

3.2 Pattern

Pattern	Description
0- UpdateDisplay	Update and display
1- Update	Update without displaying
2- Display	Display
3- LED	Flash LED
4- Key	Update key (see ESL Key)ESL Key 2.12.1 ESL Key
5- ClearGif	Clear animation (DSL only)
6- ClearGifUpdateImage	Clear animation and update static image (DSL only)

3.3 PageIndex

Page ID	Description
0- P0	Page 1
1- P1	Page 2

2- P2	Page 3
3- P3	Page 4
4- P4	Page 5; compressed image must not exceed 32 KB
5- P5	Page 6; compressed image must not exceed 32 KB
6- P6	Page 7; compressed image must not exceed 32 KB
7- P7	Page 8; compressed image must not exceed 32 KB

Note: All current ESLs have an 8-page data cache. Due to storage limitations, images sent to pages 5-8 must not exceed 32 KB after compression (LZSS compression is implemented internally by the AP; the server still sends the original image).

3.4 RGB Color Definition

The ESL LED uses RGB components and can visually produce seven colors.

Color	R	G	B
Off	×	×	×
Blue	×	×	√
Green	×	√	×
Cyan	×	√	√
Red	√	×	×
Purple	√	×	√
Yellow	√	√	×
White	√	√	√

3.5 Data Definitions (C# Version)

eStationInfor: Defines eStation configuration, status, parameters, and other information. ID, MAC, AppVersion, DummyVersion, TotalCount, and SendCount are read-only. Its definition is as follows:

```
/// <summary>
/// AP information
/// </summary>
[MessagePackObject]
public class eStationInfor
{
    /// <summary>
    /// ID: Code4
    /// </summary>
    [Key(0)]
    public string ID { get; set; } = "0000";
    /// <summary>
    /// Alias
    /// </summary>
    [Key(1)]
    public string Alias { get; set; } = "";
    /// <summary>
    /// Local IP
    /// </summary>
}
```

```
[Key(2)]
public string IP { get; set; } = "127.0.0.1";
/// <summary>
/// MAC, refer ID
/// </summary>
[Key(3)]
public string MAC { get; set; } = "90A9F7300000";
/// <summary>
/// AP type
/// </summary>
[Key(4)]
public int ApType { get; set; } = 6; // AP05 = 3;
/// <summary>
/// Firmware version
/// </summary>
[Key(5)]
public string ApVersion { get; set; } = "1.0.1";
/// <summary>
/// MOD ModVersion
/// </summary>
[Key(6)]
public string ModVersion { get; set; } = "";
/// <summary>
/// Disk size (MB)
/// </summary>
[Key(7)]
public int DiskSize { get; set; } = 0;
/// <summary>
/// Disk free space (MB)
/// </summary>
[Key(8)]
public int FreeSpace { get; set; } = 0;
/// <summary>
/// Server address
/// </summary>
[Key(9)]
public string Server { get; set; } = "192.168.4.92";
/// <summary>
/// Connection parameters
/// </summary>
[Key(10)]
public string[] ConnParam { get; set; } = Array.Empty<string>();
/// <summary>
/// Auto network
/// </summary>
[Key(11)]
public bool AutoIP { get; set; } = true;
/// <summary>
/// Local IP, empty if auto network
/// </summary>
[Key(12)]
public string LocalIP { get; set; } = string.Empty;
/// <summary>
/// Subnet mask, empty if auto network
/// </summary>
```

```
[Key(13)]
public string Subnet { get; set; } = string.Empty;
/// <summary>
/// Gateway, empty if auto network
/// </summary>
[Key(14)]
public string Gateway { get; set; } = string.Empty;
/// <summary>
/// Heartbeat speed
/// </summary>
[Key(15)]
public int Heartbeat { get; set; } = 15;
}
```

eStationConfig: Defines AP configuration information. Its definition is as follows:

```
[MessagePackObject]
internal class eStationConfig
{
    /// <summary>
    /// Alias
    /// </summary>
    [Key(0)]
    public string Alias { get; set; } = "";
    /// <summary>
    /// Server address
    /// </summary>
    [Key(1)]
    public string Server { get; set; } = "192.168.4.92";
    /// <summary>
    /// Connection parameters
    /// </summary>
    [Key(2)]
    public string[] ConnParam { get; set; } = Array.Empty<string>();
    /// <summary>
    /// Auto network
    /// </summary>
    [Key(3)]
    public bool AutoIP { get; set; } = true;
    /// <summary>
    /// Local IP, empty if auto network
    /// </summary>
    [Key(4)]
    public string LocalIP { get; set; } = string.Empty;
    /// <summary>
    /// Subnet mask, empty if auto network
    /// </summary>
    [Key(5)]
    public string Subnet { get; set; } = string.Empty;
    /// <summary>
    /// Gateway, empty if auto network
    /// </summary>
    [Key(6)]
    public string Gateway { get; set; } = string.Empty;
    /// <summary>
    /// Heartbeat speed
```



```

    /// </summary>
    [Key(7)]
    public int Heartbeat { get; set; } = 15;
}

```

eStationMessage: Defines AP message information. Its definition is as follows:

```

[MessagePackObject]
public class eStationMessage
{
    /// <summary>
    /// Message code
    /// </summary>
    [Key(0)]
    public MessageCode Code { get; set; } = MessageCode.OK;
}

```

ESLEntity: Defines ESL task data. Its definition is as follows:

```

    /// <summary>
    /// ESL entity
    /// </summary>
    [MessagePackObject]
    public class ESLEntity
    {
        [Key(0)]
        public string TagID { get; set; }
        [Key(1)]
        public Pattern Pattern { get; set; }
        [Key(2)]
        public PageIndex PageIndex { get; set; }
        [Key(3)]
        public bool R { get; set; }
        [Key(4)]
        public bool G { get; set; }
        [Key(5)]
        public bool B { get; set; }
        [Key(6)]
        public int Times { get; set; }
        [Key(7)]
        public int Token { get; set; }
        [Key(8)]
        public string OldKey { get; set; }
        [Key(9)]
        public string NewKey { get; set; }
        [Key(10)]
        public string Base64String { get; set; } = "";
    }

```

TaskResult: Defines the communication results for a group of ESL/PTL devices. Its definition is as follows:

```

    /// <summary>
    /// Task result entity
    /// </summary>
    [MessagePackObject]
    public class TaskResult
    {
        /// <summary>

```

```

/// COM Port
/// </summary>
[Key(0)]
public int Port { get; set; } = 0;

/// <summary>
/// Total count in cache
/// </summary>
[Key(1)]
public int TotalCount { get; set; }

/// <summary>
/// Current sending count
/// </summary>
[Key(2)]
public int SendCount { get; set; }

/// <summary>
/// Message
/// </summary>
[Key(3)]
public MessageCode Message { get; set; } = 0;

/// <summary>
/// Tag results list
/// </summary>
[Key(4)]
public List<TagResult> Tags { get; set; } = new List<TagResult>();
}

```

TagResult: Defines the communication result for a single ESL/PTL device. Its definition is as follows:

```

[MessagePackObject]
/// <summary>
/// Tag result
/// </summary>
public class TagResult
{
    /// <summary>
    /// Tag ID
    /// </summary>
    [Key(0)]
    public string TagID { get; set; } = string.Empty;

    /// <summary>
    /// RF power
    /// </summary>
    [Key(1)]
    public int RfPower { get; set; } = -256;

    /// <summary>
    /// Battery
    /// </summary>
    [Key(2)]
    public byte Battery { get; set; } = 0;

    /// <summary>
    /// Version

```

```
/// </summary>
[Key(3)]
public byte Version { get; set; } = 0;
/// <summary>
/// Status
/// </summary>
[Key(4)]
public byte Status { get; set; } = 0;
/// <summary>
/// Token
/// </summary>
[Key(5)]
public int Token { get; set; } = 0;
/// <summary>
/// Temperature
/// </summary>
[Key(6)]
public int Temperature { get; set; } = 0;
/// <summary>
/// Channel
/// </summary>
[Key(7)]
public int Channel { get; set; } = 0;
/// <summary>
/// [DSL]UTC time,
/// </summary>
[Key(8)]
public byte[] UtcTime { get; set; } = Array.Empty<byte>();
/// <summary>
/// [DSL]UTC percent,
/// </summary>
[Key(9)]
public byte[] TimePercent { get; set; } = Array.Empty<byte>();
/// <summary>
/// [DSL]Count
/// </summary>
[Key(10)]
public byte Count { get; set; } = 0;
}
```

